

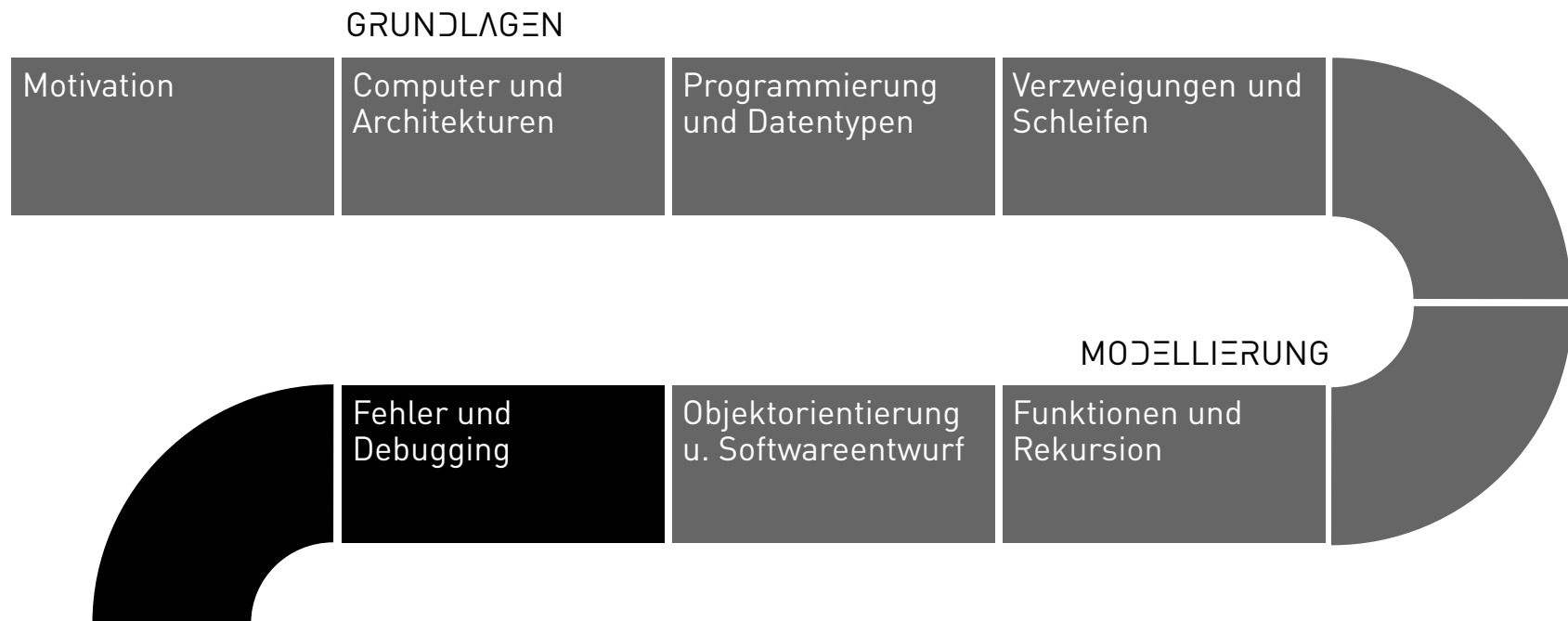
Programming and databases

Joern Ploennigs

Algorithms

MIDJOURNEY: TWO TOWERS, REF. M.C. ESCIER

PROCESS



ALGORITHMS



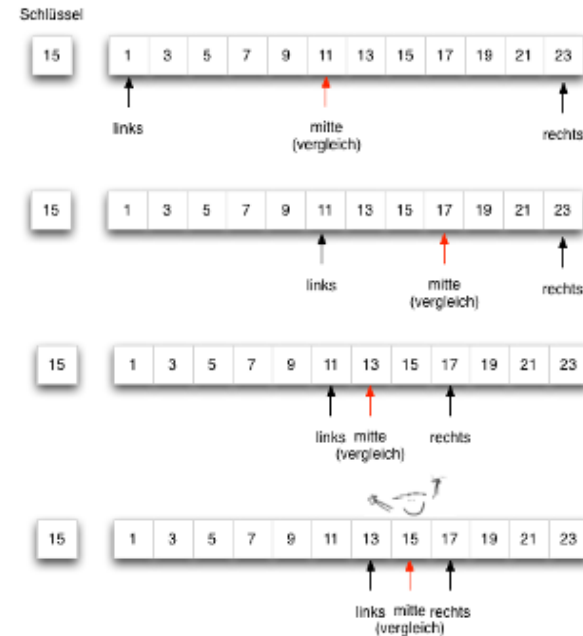
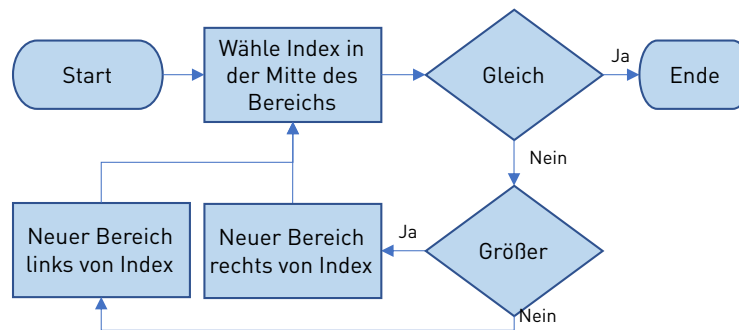
Definition: Algorithms

Algorithms are well-defined, unambiguous instructions.

- Known examples:
 - Long division
 - Triangle construction
- In computer science practice: computer-executable and terminating
- Programming means combining algorithms and data structures in such a way that you obtain the desired input to the desired output.

EXAMPLE FOR DIVIDE AND CONQUER – BINARY SEARCH

- Find the index of a number in a list (e.g., to check whether the number occurs in a set)
- Instead of searching the entire list, you split the list recursively into two parts and check whether the searched number is equal (found), greater (to the right), or smaller (to the left) in the list

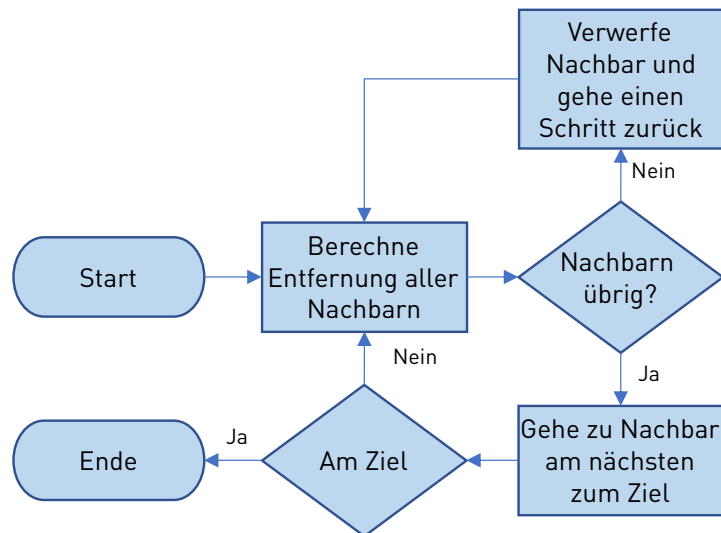
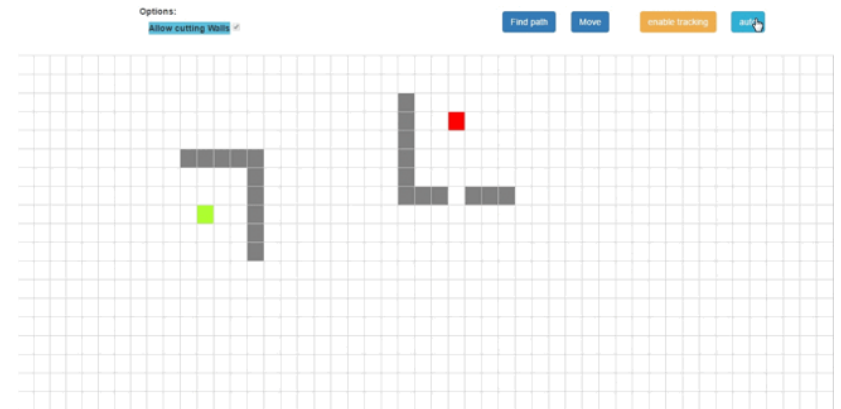


HEURISTICS

- They are not based on well-defined or unambiguous steps, but on practical experience
- They aim to quickly achieve a sufficiently accurate result. There is *no* guarantee to find the optimal solution.
- Example:
 - An expert's estimate
 - Stepwise approximation

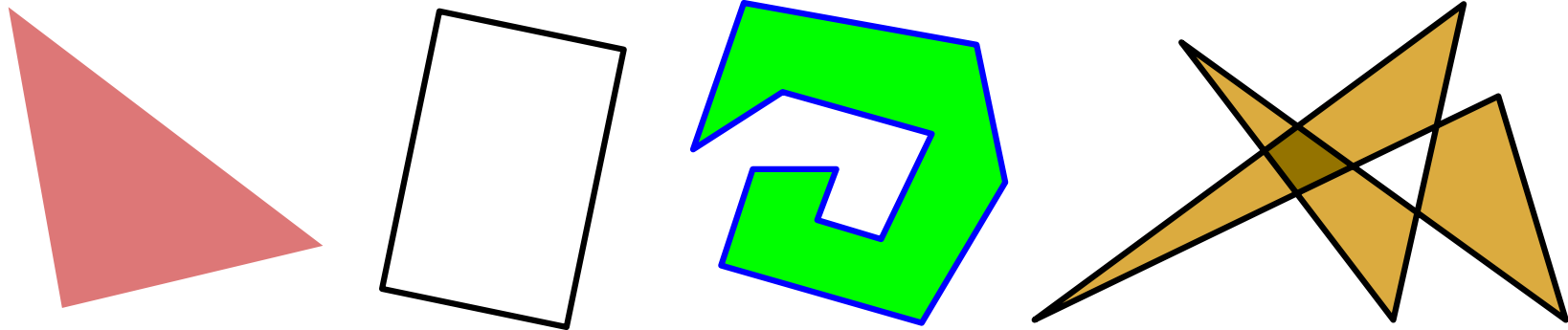
EXCURSUS — EXAMPLE OF A HEURISTIC: THE A* SEARCH ALGORITHM

- Pathfinding is one of the most complex problems in computer science (NP-complete: due to the combinatorial nature of the problem)
- A* is a heuristic for finding the shortest path between two points in a grid with obstacles



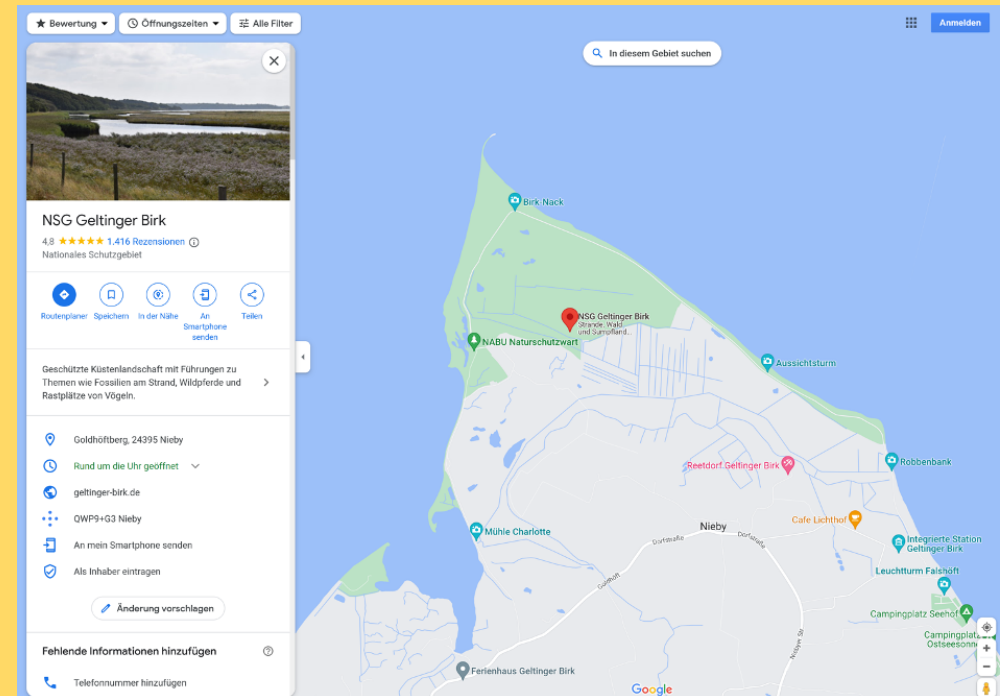
EXAMPLE PROBLEM - Is a point X INSIDE A POLYGON?

- A polygon is an arbitrary polygon
- The solution to this problem is a common part of calculations in civil engineering and environmental sciences, e.g., to test whether a proposed project lies within a nature reserve.



LECTURE HALL QUESTION

How can I determine whether a point lies inside a polygon?



Google Maps

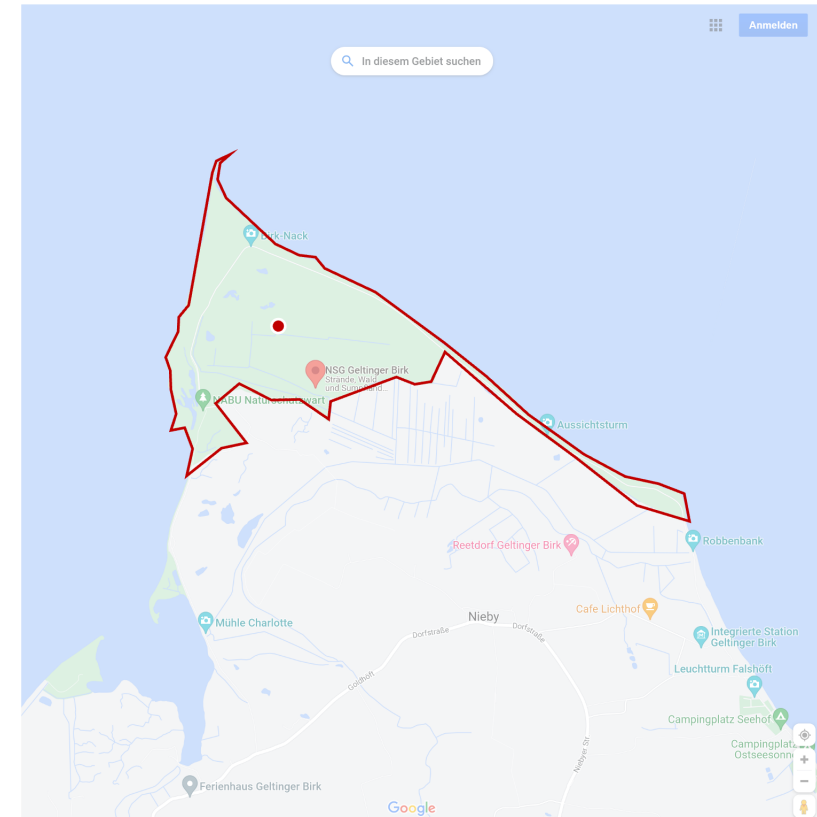
EXAMPLE PROBLEM - WHAT ARE OUR INPUTS AND OUTPUTS?

Inputs:

- The position of the point x
- A polyline describing the boundary of the polygon
 - A list of connected, consecutive lines

Outputs:

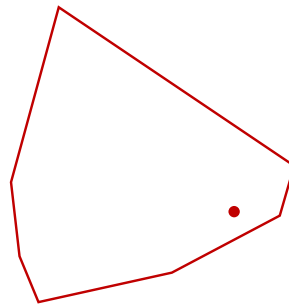
- A Boolean



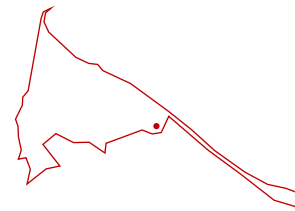
Google Maps

EXAMPLE PROBLEM - DESIGNING AN ALGORITHM

- What computations can be performed on points and lines?
- We can check whether a point lies exactly on a line.
- We can check on which side of a line a point lies.
- Is Point 2 sufficient? For convex shapes – yes; if the point lies on the same side of all lines of the polygon, it lies inside the polygon.
- But this does not hold for concave shapes => it is thus a heuristic!



Convex polygon



Concave polygon

EXAMPLE PROBLEM - DESIGNING AN ALGORITHM

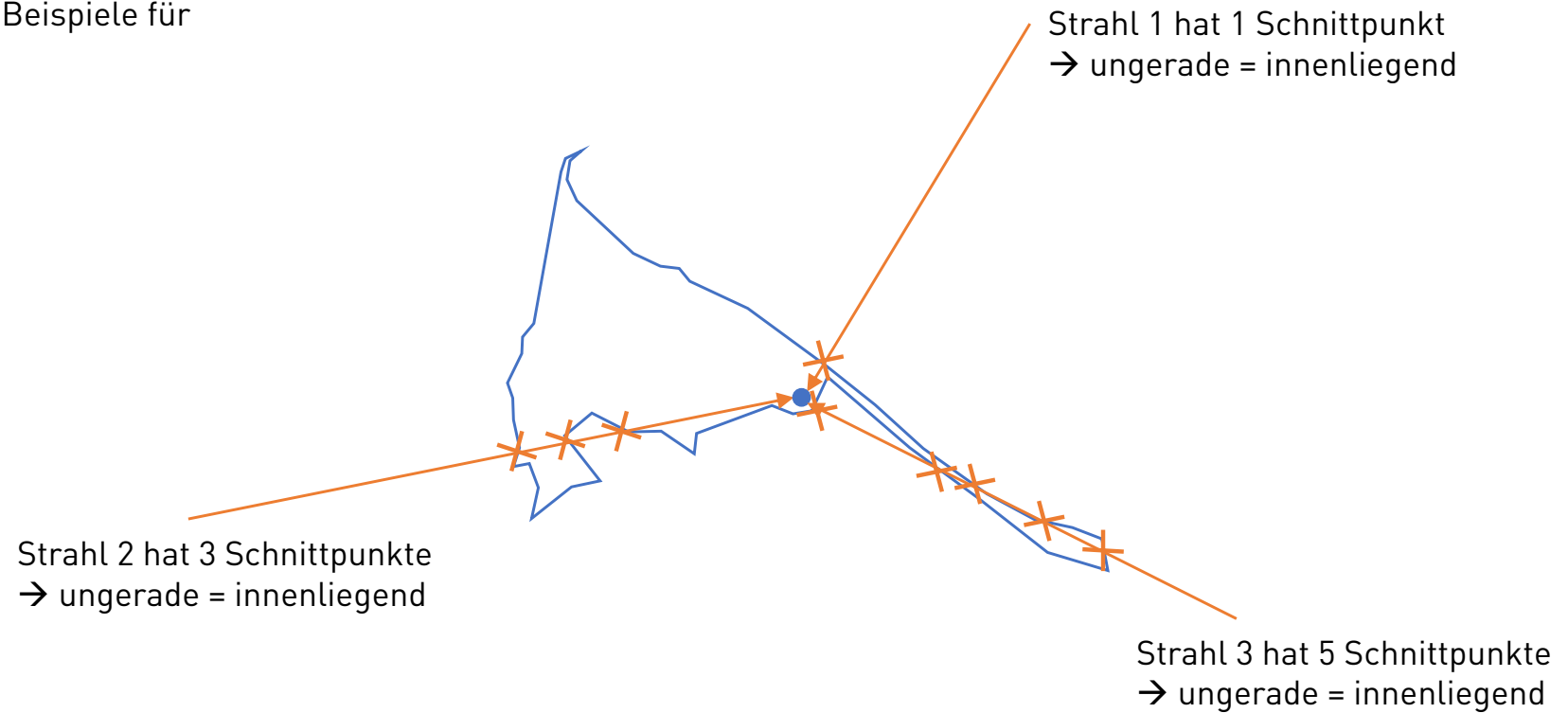
- Designing often means looking at a problem from different angles and finding new computational perspectives.
- Here: The crux is whether a region is concave or convex. One determines these properties by drawing lines and performing intersection tests.
- Is the key to this problem therefore in the intersection tests between lines?
- Is there a line from the point in question **X** to another point **Y**, whose intersections with the polygon's edges provide an answer to the question?

EXAMPLE PROBLEM - DESIGNING AN ALGORITHM

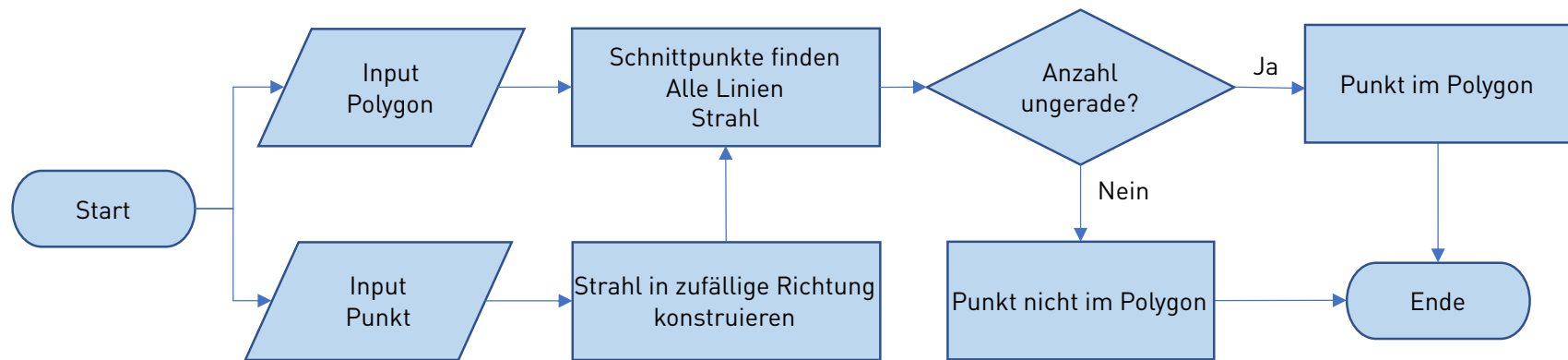
- The answer: Any point outside the polygon!
- If the point **x** lies inside the polygon, the drawn line will intersect the boundary at least once.
- If the drawn line passes randomly through a concave region, there will be two more intersection points, for a total of 3 intersection points, or 5, or ...
- If the point **x** lies outside, there will thus be either no intersection points, or 2, or 4, or ...
- **Thus the problem can be reduced to the number of intersection points!**

EXAMPLE PROBLEM — EXAMPLE

Beispiele für



EXAMPLE PROBLEM - DRAW AS A PROGRAM FLOWCHART



Examples for:

- Ray 1 has 1 intersection point → odd = inside
- Ray 2 has 3 intersection points → odd = inside
- Ray 3 has 5 intersection points → odd = inside

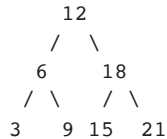
RECURSION - EXAMPLE: TREE TRAVERSAL

Tree: A data structure in which each element can refer to further subelements.

Now we want to apply a function to every element in the tree.

Simplest representation of a tree:

- A tuple with a value and a list
- This list contains tuples, each with a value and a list (a recursive data structure!)



```
baum = (12, [  
        (6, [  
            (3, []),  
            (9, [])  
        ]),  
        (18, [  
            (15, []),  
            (21, [])  
        ])  
    ])
```

RECURSION - EXAMPLE: TREE TRAVERSAL

Traversals are functions that traverse the tree, e.g., to search for elements.

CS ASIDE: SORTING

- Specifically the sorting of lists, regardless of data type
- One of the standard use cases for loops and recursion
- The go-to solution in everyday programming: `sorted()`
- But: a good example of a structured computer science problem

SORTING - ALGORITHM 1: BUBBLE SORT

- Each element in the list is traversed and compared with the next element.
- If the next element is smaller than the current one, swap their positions.
- The list is repeatedly traversed until no swaps occur.

```
def bubbleSort(numbers):  
    for i in range(len(numbers)-1):  
        for j in range(0, len(numbers)-i-1):  
            if numbers[j] > numbers[j + 1]:  
                numbers[j], numbers[j + 1] = numbers[j + 1], numbers[j]
```

SORTING - ALGORITHM 2: QUICK SORT

The basic principle is partitioning the list:

1. Store the first element of the list as the pivot element.
2. Then traverse the list and compare each element with the pivot.
3. Insert into one of three lists: Smaller, Equal, and Greater.
4. Then Quick Sort is applied recursively to the Smaller and Greater lists.
5. In the end, all the lists are recursively merged back together.

COMPUTER SCIENCE DIGRESSION: SEARCHING

- Specifically searching on sorted lists and trees
- Most of the time you know the number of elements, so you know exactly where the middle is, for example.
- Later lectures: In real-world applications, datasets often have more than one value (tables instead of lists); here one typically works with indexing.

SEARCHES - SEARCHING IN LISTS

The simplest method: *linear search*

```
def contains(list, x):  
    for l in list:  
        if l == x:  
            return True  
    return False
```

SEARCHING - SEARCHING IN LISTS

The optimal method: *Binary search*

- Significantly more complex code
- *Idea*: Find the middle element of the list, then compare it with the target value
- If the value is smaller: use the same procedure for the first half of the list
- If the value is larger: use the same procedure for the second half of the list
- The implementation is usually via recursion

SEARCH - SEARCHING IN TREES

Strategies for searching an unordered tree:

- *Breadth-first search*: Beginning at the root, traverse each level of the tree from left to right.
- *Depth-first search*: Follow a path from the root to the end, then backtrack step by step until more paths become available.

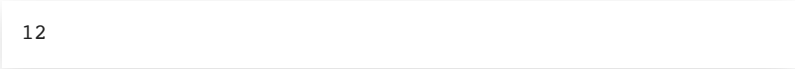
SEARCHING - SEARCHING IN TREES

- In sorted trees (search trees), the result can be found extremely efficiently, since only a single path needs to be traversed.
- *How do you sort a tree?*

CREATING A BINARY SEARCH TREE

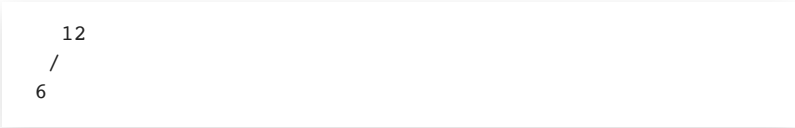
Initial list: 12 6 18 15 3 21 9

Step 1: Set 12 as the root



12

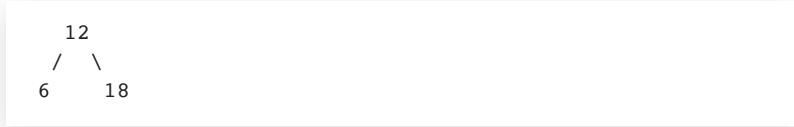
Step 2: 6 < 12, insert on the left



```
graph TD; 12 --> 6
```

12
/
6

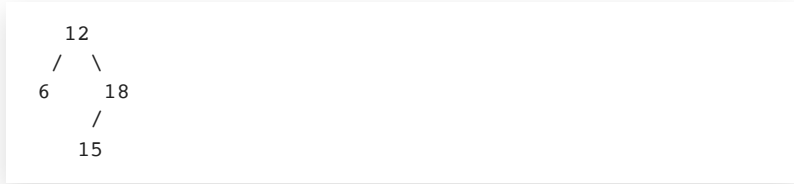
Step 3: 18 > 12, insert on the right



```
graph TD; 12 --> 6; 12 --> 18
```

12
/ \
6 18

Step 4: 15 > 12, but 15 < 18

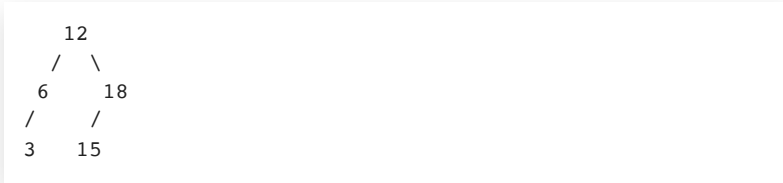


```
graph TD; 12 --> 6; 12 --> 18; 18 --> 15
```

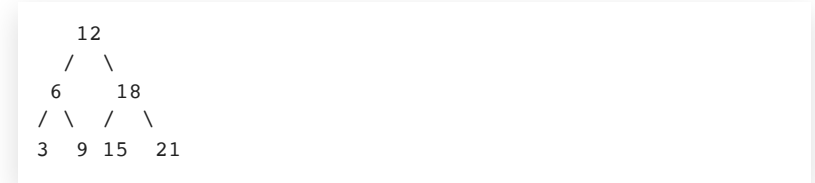
12
/ \
6 18
 /
 15

SEARCH TREE - CONTINUATION

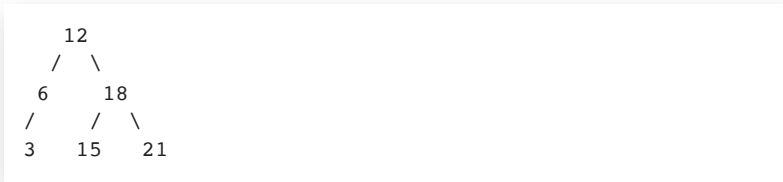
Step 5: $3 < 12$ and $3 < 6$



Step 7: $9 < 12$ but $9 > 6$



Step 6: $21 > 12$ and $21 > 18$



Finished search tree!

AS AN ASIDE: COMPLEXITY AND EFFICIENT ALGORITHMS

- The actual computation time is determined only by the exact input data and the processor used.
- Formally, efficiency is better described as an increasing curve that provides a relative estimate of how much longer an algorithm must run as the input data grows.
- *Notation:* $O(x)$, where x denotes the amount of necessary computation steps.
- For trees and lists, this depends on the number of elements (variable n).

COMPLEXITY - COMPARISON OF ALGORITHMS

Search algorithms:

- Linear search

$$O(n)$$

- Binary search

$$O(\log(n))$$

Sorting algorithms:

- Bubble Sort

$$O(n^2)$$

- Quick Sort

$$O(n \cdot \log(n))$$

Questions?

